

Revisão para a Prova Final

Banco de Dados Não Relacional – Prof.^a Lucineide

Aluna: Larissa Candido

PARTE A – QUESTÕES OBJETIVAS

Questão 1: b) Embedding

Questão 2: c) \$group

Questão 3: c) Projeção

Questão 4: c) skip() e limit()

Questão 5: b) \$exists

PARTE B – QUESTÕES DISCURSIVAS

Questão 1:

A técnica utilizada é o **Embedding**, que consiste em armazenar informações relacionadas dentro do mesmo documento. A principal vantagem dessa abordagem é a rapidez na leitura dos dados, pois todas as informações necessárias ficam concentradas em um único documento. Isso reduz a necessidade de consultas adicionais. Como limitação, o documento pode crescer excessivamente ao longo do tempo, dificultando a manutenção e impactando o desempenho quando houver um grande volume de histórico armazenado.

Questão 2:

Utilizar coleções separadas para clientes e vendedores com referências evita a duplicação de dados. Caso um vendedor altere alguma informação, a atualização ocorre apenas em um único documento. Essa abordagem facilita a manutenção, melhora a consistência dos dados e reduz o espaço de armazenamento utilizado. Além disso, torna o sistema mais organizado e preparado para crescer sem gerar redundância excessiva.

Questão 3:

O Aggregation Framework permite processar e resumir grandes quantidades de dados diretamente no MongoDB. Para gerar relatórios de leads por origem ou vendedor, pode-se utilizar o operador **\$group** para agrupar os documentos. O operador **\$match** pode filtrar registros específicos antes do agrupamento. Já o **\$project** pode selecionar apenas os campos necessários para exibição do relatório. Dessa forma, o MongoDB produz estatísticas de forma eficiente.

Questão 4:

Índices são estruturas utilizadas para acelerar consultas no banco de dados. Eles funcionam de maneira semelhante ao índice de um livro, permitindo localizar informações sem percorrer todos os registros. Quando um índice é criado sobre determinado campo, o MongoDB consegue encontrar os documentos desejados mais rapidamente. Os principais benefícios são a redução do tempo de resposta das consultas e a melhoria do desempenho geral do sistema.

Questão 5:

A duplicação de clientes pode ocorrer quando informações são armazenadas repetidamente em vários documentos. Uma modelagem adequada reduz esse problema utilizando referências e identificadores únicos. Dessa forma, cada cliente é armazenado apenas uma vez, sendo referenciado quando necessário. Isso melhora a consistência dos dados, facilita atualizações futuras e reduz o risco de informações divergentes dentro do sistema.

Questão 6:

Armazenar clientes, vendedores, leads e negociações em uma única coleção possui como vantagem a simplicidade inicial do desenvolvimento. Entretanto, essa abordagem dificulta consultas específicas, aumenta a quantidade de dados desnecessários em cada documento e torna a manutenção mais complexa. No futuro, o crescimento da aplicação pode gerar problemas de desempenho e organização. Por isso, normalmente é mais adequado separar os dados em coleções distintas.

Questão 7:

A paginação é importante para evitar o carregamento excessivo de registros em uma única consulta. Isso melhora o desempenho da aplicação e proporciona uma melhor experiência ao usuário. O comando **skip()** define quantos documentos devem ser ignorados antes do início da exibição dos resultados. Já o comando **limit()** determina quantos registros serão retornados por página. Juntos, permitem navegar pelos dados de forma eficiente.

Questão 8:

A ausência de um esquema rígido oferece grande flexibilidade para armazenar documentos com estruturas diferentes. Isso facilita a evolução do sistema e a adaptação a novas necessidades de negócio. Entretanto, a equipe deve estabelecer padrões para evitar inconsistências entre documentos. Também é importante realizar validações na aplicação para garantir a qualidade e a integridade das informações armazenadas.

Questão 9:

O **Embedding** apresenta melhor desempenho de leitura, pois os dados relacionados ficam armazenados juntos. Porém, pode gerar documentos muito grandes e aumentar a duplicação de informações. Já o **Referencing** reduz redundâncias e facilita a manutenção, pois os dados são armazenados separadamente. Em termos de escalabilidade, referências costumam ser mais adequadas para relacionamentos complexos, enquanto embedding funciona melhor para informações fortemente relacionadas e acessadas em conjunto.

Questão 10:

A integração entre MongoDB e Node.js oferece comunicação eficiente utilizando dados em formato JSON, o que simplifica o desenvolvimento. O Node.js permite criar APIs rápidas para acesso ao banco de dados. Essa integração facilita operações CRUD, consultas avançadas e atualizações em tempo real. Para os usuários finais, isso resulta em maior velocidade, melhor desempenho e uma experiência mais fluida na utilização do sistema.

Questão 11:

Uma estratégia eficiente consiste em utilizar o Aggregation Framework. O operador **\$group** pode agrupar as negociações pelo identificador do vendedor, enquanto **\$sum** contabiliza a quantidade de negociações concluídas. Em seguida, o operador **\$sort** pode ordenar os resultados do maior para o menor valor. Dessa forma, a equipe comercial consegue identificar rapidamente quais vendedores apresentam melhor desempenho.

Questão 12:

Para garantir escalabilidade, é importante definir corretamente as coleções e evitar duplicação excessiva de dados. O uso adequado de índices melhora a velocidade das consultas. A escolha entre embedding e referencing deve considerar o volume e a frequência de acesso às informações. Além disso, a estrutura flexível do MongoDB permite adicionar novos campos sem necessidade de grandes alterações, facilitando o crescimento da aplicação.

Questão 13:

O uso de embedding pode ser adequado quando o histórico de interações não cresce excessivamente e normalmente é acessado junto com os dados do cliente. Essa abordagem simplifica consultas e melhora o desempenho de leitura. Entretanto, se o histórico se tornar muito grande ao longo do tempo, o documento poderá aumentar significativamente. Nesse cenário, o uso de referências pode ser mais adequado para manter a escalabilidade do sistema.

Questão 14:

Os bancos relacionais utilizam tabelas, linhas e colunas com esquemas rígidos. Já o MongoDB utiliza documentos JSON com estrutura flexível. Em termos de flexibilidade, o MongoDB facilita alterações e evolução do sistema sem grandes modificações estruturais. Também apresenta melhor escalabilidade horizontal para grandes volumes de dados. Por outro lado, bancos relacionais costumam ser mais adequados para cenários com relacionamentos complexos e forte necessidade de integridade transacional.

Questão 15:

A adoção do MongoDB pode ser justificada por diversos fatores. Primeiro, sua estrutura flexível permite adaptar facilmente os documentos às necessidades do negócio. Segundo, sua escalabilidade horizontal possibilita lidar com o crescimento do volume de dados da empresa. Terceiro, o formato JSON facilita a integração com aplicações modernas desenvolvidas em Node.js. Além disso, o MongoDB oferece recursos avançados de agregação, indexação e desempenho para consultas em ambientes de grande escala.